Programação 2

Aula 12 -
Boas Práticas: padrões qualidade e estilo de codificação

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

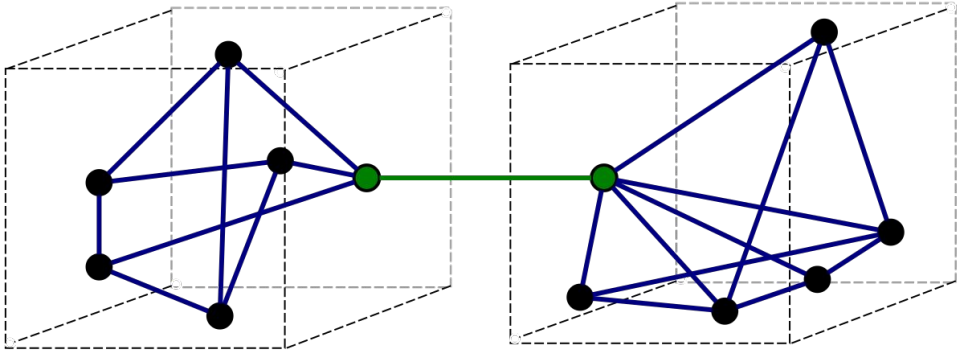
Coesão e Acoplamento

Coesão

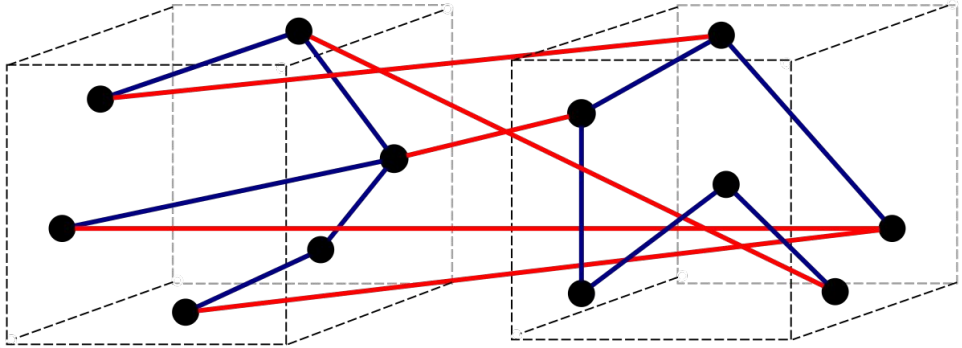
Se refere ao relacionamento que os membros de um módulo possuem. Códigos coesos são aqueles de relação forte, onde seus **membros estão intimamente ligados e estão ali por um objetivo comum**. Membros que não são absolutamente necessários para aquele módulo não devem estar presentes em códigos coesos.

Acoplamento

Se refere ao relacionamento entre os módulos. Indica **quanto um módulo depende de outro** para funcionar. Códigos desacoplados são aqueles de relação fraca, e não dependem de outros para fazer sua funcionalidade básica. É sempre desejável um baixo nível de acoplamento.



a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

[java - O que são os conceitos de coesão e acoplamento? - Stack Overflow em Português]

Modificadores de Acesso: **public** vs **private**

public

indica que aquele elemento (ex.: classe, atributo, método...)
é acessível dentro e fora daquela classe

private

indica que aquele elemento (ex.: classe, atributo, método...)
é acessível somente dentro daquela classe

é uma forma de proteger/limitar o acesso

uma boa prática de programação é sempre usar *private* como
padrão e somente tornar algo *public* quando necessário

Métodos de Acesso e Atualização: **get** , **set**

get

É comum se usar um método *get* para acessar uma variável. O uso deste tipo de recurso possibilita garantir que aquela variável não será alterada de forma indevida.

set

O método *set* é comumente utilizado na definição das classes para garantir que o caminho de alteração das variáveis da instância sempre ocorram por um único fluxo.

Conceito de Hoje

Boas Práticas

Google Style Guide

Google Java Style Guide

Table of Contents

1 Introduction
- 1.1 Terminology notes
- 1.2 Guide notes

2 Source file basics
- 2.1 File name
- 2.2 File encoding: UTF-8
- 2.3 Special characters

3 Source file structure
- 3.1 License or copyright information, if present
- 3.2 Package statement
- 3.3 Import statements

- 4.6 Whitespace
- 4.7 Grouping parentheses: recommended
- 4.8 Specific constructs

5 Naming
- 5.1 Rules common to all identifiers
- 5.2 Rules by identifier type
- 5.3 Camel case: defined

6 Programming Practices
- 6.1 @Override: always used
- 6.2 Caught exceptions: not ignored
- 6.3 Static members: qualified using class

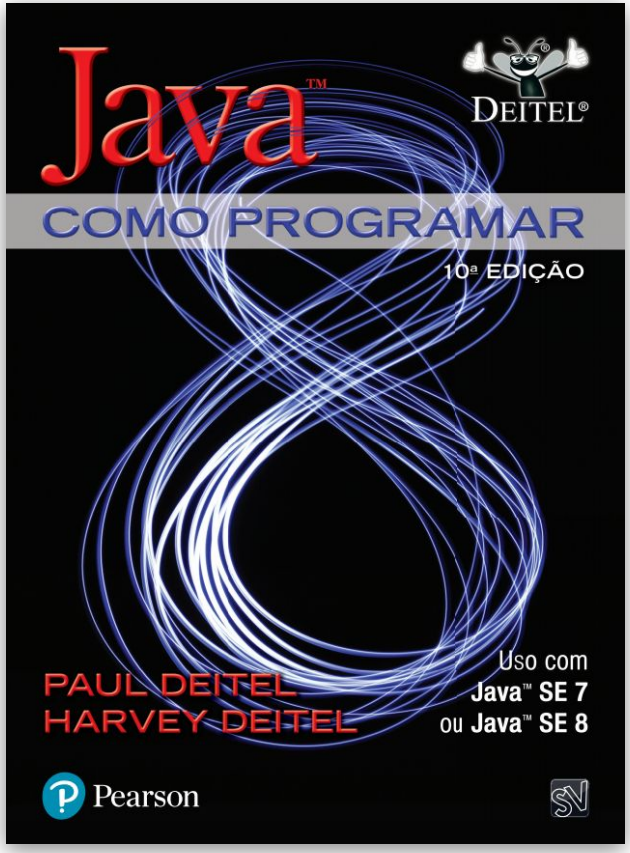
https://google.github.io/styleguide/javaguide.html

Mini-Projeto

Refatoração do código.

Padronize seu código a partir das seções especificadas do guia de estilo de Java do Google (https://google.github.io/styleguide/javaguide.html)

- Seção 2.1. Nomes dos arquivos fonte (.java e .fxml)
- Seção 4.2. Indentação de blocos.
- Seção 4.4. Limite da coluna
- Seção 5. Nomeação.
- Seção 7.3. Documentação em Java.

Referências

Java - Como Programar (10a Edição)